C. Bracket Sequence Deletion

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1200 | Tags: greedy, implementation | https://codeforces.com/problemset/problem/1657/C

You are given a bracket sequence consisting of n characters '(' and/or ')'. You perform several operations with it.

During one operation, you choose the **shortest** prefix of this string (some amount of first characters of the string) that is **good** and remove it from the string.

The prefix is considered **good** if one of the following two conditions is satisfied:

- this prefix is a regular bracket sequence;
- this prefix is a palindrome of length **at least two**.

A bracket sequence is called regular if it is possible to obtain a correct arithmetic expression by inserting characters '+' and '1' into this sequence. For example, sequences `(()())()`, `()` and `(()(()))` are regular, while `)(`, `(()` and `(()))(` are not.

The bracket sequence is called palindrome if it reads the same back and forth. For example, the bracket sequences `))`, `((` and `)(()` are palindromes, while bracket sequences `()`, `)(` and `))(` are not palindromes.

You stop performing the operations when it's not possible to find a **good** prefix. Your task is to find the number of operations you will perform on the given string and the number of remaining characters in the string.

You have to answer t independent test cases.

Input

The first line of the input contains one integer t ($1 \le t \le 10^4$) — the number of test cases. The next $2t$ lines describe test cases.

The first line of the test case contains one integer n ($1 \le n \le 5 \cdot 10^5$) — the length of the bracket sequence.

The second line of the test case contains n characters '(' and/or ')' — the bracket sequence itself.

It is guaranteed that the sum of n over all test cases do not exceed $5 \cdot 10^5$ ($\sum n \le 5 \cdot 10^5$).

Output

For each test case, print two integers c and r — the number of operations you will perform on the given bracket sequence and the number of characters that remain in the string after performing all operations.

Example

Input

```
5
2
()
3
())
4
((((
5
)(((()
6
)((()(
```

Output

```
1 0
1 1
2 0
1 0
1 1
```